

Yesterday: "Can I determine whether Alice successfully or unsuccessfully authenticated?"

Today: "If somebody changes AD itself, can I determine what changed, who changed it, and when?"

Imagine someone: creates a suspicious user, adds them to an important security group, removes them later, or deletes the account. Even if the account disappears, the logs can preserve evidence of what occurred.

By the end of this 2-hour session:

- ☐ Pass a short closed-book review
- ☐ Understand logon auditing vs account-management auditing
- ☐ Understand actor vs target
- ☐ Determine where AD identity-change logs belong
- ☐ Verify/configure appropriate auditing on the DC
- ☐ Create one controlled temporary user
- ☐ Add/remove that user from a group
- ☐ Delete the user
- ☐ Discover the resulting Windows events
- ☐ Use Get-WinEvent again without copying yesterday
- ☐ Build a short identity-change timeline
- ☐ Explain what the evidence proves

Closed Book Review:

4624 means?

Successful authentication.

4625 means?

Failed authentication.

4688 means?

New process created.

Logon Type 2: Interactive, Sitting at the computer, enter with kb+m

Logon Type 3: Network

Logon Type 10: Remote

Logon Type 11: Previously remembered user on the workstation

\$env:LOGONSERVER tells you what?

Tells you which domain controller authenticated the logon.

Nltest /dsgetdc:kevinlab.local tells you what?

Tells Windows to locate the Domain Controller for that domain and provides information about it.

What PowerShell cmdlet reads Windows Event Logs?

Get-WinEvent

Why might CLIENT01 and SRV-DC01 record different evidence?

CLIENT01 has event logs of what is happening on the local machine, while SRV-DC01 has event logs of what the Domain Controller is doing.

Authentication vs Authorization:

Am I who I claim to be vs once authenticated....what permissions do I have?

Learn Today's Mental Model

Yesterday involved an existing identity being used:

Ahr already exists -> ahr attempts login -> successful/failed authentication -> 4624/4625

Today involves changing the identity system itself:

Administrator -> Creates Account -> Changes Group Membership -> Changes Authorization -> Deletes Account

Logon Auditing: Answers things like: Did Alice authenticate?

Account Management Auditing: Answers questions like: Did somebody create Alice? Did somebody change Alice's account?

Security Group Management Auditing: Answers: Did somebody add Alice to a security group?

Actor vs Target: Suppose KEVINLAB\Administrator adds audittest to GG_HR_Users

The Actor / Subject: **KEVINLAB\Administrator** "Who performed the action?"

The Target / Member: **audittest** "Who was affected?"

The Affected Group: **GG_HR_Users** "What authorization structure changed?"

Challenge 1: Prepare the Domain Controller

Objective: Determine whether SRV-DC01 is configured to record user-account and security-group management events.

Which machine should primarily record changes made to domain accounts?

SRV-DC01. Recorded on the Domain Controller, not on the local client.

Would yesterday's audit logon policy logically cover: Creating a new domain account?

No. Logon auditing is about signins.

Explore: Advanced Audit Policy Configuration. Find categories that logically correspond to: user account creation/deletion or security group changes.

You currently have: GPO-Workstation-Security-Auditing. Would you apply Domain Controller security settings through the Workstation GPO? Or would something like GPO-DomainController-Security-Auditing linked to: Domain Controllers OU be cleaner?

The ladder.

Task: Figure out

1. Relevant auditing category/subcategory
 - a. **Within account management inside audit policies, the two should be Audit User Account Management and Audit Security Group Management.**
2. Whether it's currently enabled
 - a. **Currently, not configured.**
3. What should be audited: success, failure, or both.
 - a. **I did success for both.**
4. How to apply the policy to the DC.
 - a. **Gpupdate /force, check note to self.**
5. How to prove the DC received it.
 - a. **Gpresult /r**

Note to self: There's a normal schedule for Group Policy refreshing. (Every 90 mins + offset by default...?)

Challenge 2: Generate Known Activity

Create a temporary account...Display Name: Audit Test....Username: audittest

Self-note: to confirm existence of account (**Get-ADUser username**)

Scenario: create audittest -> add audittest to GG_HR_Users -> remove audittest from GG_HR_Users => Delete audittest

Actor: KEVINLAB\Administrator

Target: audittest

Group: GG_HR_Users

Machine I expect evidence on: SRV-DC01

Reason: Temporary account needed to prove logging functionality.

Timestamps:

Account creation: 8/10/2026 11:28:19 AM (4720 EVENT ID)

Added to GG_HR_Users: 8/10/2026 11:29:04 AM (4728 EVENT ID)

Removed from GG_HR_Users: 8/10/2026 11:32:34 AM (4729 EVENT ID)

Account deleted: 8/10/2026 11:33:14 AM (4726 EVENT ID)

Challenge 3: Investigate the Incident

Now pretend someone else performed the activity.

Ticket: An unfamiliar account called **audittest** appeared in the domain and may have received access to HR resources. Determine what happened.

Event ID for account creation is 4720.

Adding an account to a security group: 4728.

Removing an account from security group: 4729.

Account deletion: 4726.

For each relevant event:

Timestamp, event ID, event description, actor/subject, target/member, affected group, what does this event prove?

The account creation:

Timestamp: 8/10/2026 at 11:28:19 AM, Event ID: 4720, Event Description: A user account was created, Actor/subject: Administrator, Target/member: audittest, Affected Group: None, Proves: An account named audittest was created by Administrator.

Added to GG_HR_Users:

Timestamp: 8/10/2026 at 11:29:04 AM, Event ID: 4728, Event Description: A member was added to a security-enabled global group. Actor/subject: Administrator, Target/member: audittest, Affected Group: GG_HR_Users, Proves: Administrator added audittest to GG_HR_Users security-enabled global group.

Removed from GG_HR_Users:

Timestamp: 8/10/2026 at 11:32:34 AM, Event ID: 4729, Event Description: A member was removed from a security-enabled global group. Actor/subject: Administrator, Target/member: audittest, Group affected: GG_HR_Users, Proves: Administrator removed audittest from GG_HR_Users security-enabled global group

Account deleted:

Timestamp: 8/10/2026 at 11:33:14 AM, Event ID: 4726, Event Description: A user account was deleted. Actor/subject: Administrator, Target/member: audittest, Group affected: None, Proves: Administrator deleted account audittest.

Powershell Investigation

Yesterday: used Get-WinEvent.

Objective: Retrieve the relevant identity-change events from the Security log without scrolling through Event Viewer.

**Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4720,4728,4729,4726}
-MaxEvents 30**

Note to self: Obviously, this alone doesn't give the info that Event Viewer would, but later on we can add more to the command in terms of filtering to get more use out of it. Not sure of the advantages of Powershell in this situation yet, and I prefer using the Event Viewer. Later on, maybe Powershell can automate things like this?

Mini Investigation Report:

The details for the timeline are above; not interested in retyping it.

Conclusion: The admin made an account for testing purposes; it wasn't given any special privileges. The sequence of events is consistent with a temporary admin test.

If the account had been unexpectedly placed into a highly privileged administrative group, the security impact would be greater.

CompTIA Security+ Block

Refer to other document.

This is now the second session of the day....I know, I want to be the best that much...lol.

Main Question: I already know that Event 4688 can tell me that: powershell.exe started. But consider this: An analyst sees that Powershell ran. Okay... What did Powershell actually execute?

4688 records process creation, but the command-line field requires an additional policy to be enabled. Microsoft also warns that command-line arguments can contain sensitive information. Powershell script block logging provides another layer: Powershell activity can generate Event ID 4104 in the Microsoft-Windows-Powershell/Operational log containing script-block content.

Learn the Visibility Problem

Level 1: Process

Yesterday: 4688 -> A new process was created -> powershell.exe

That can answer: Did Powershell launch? But suppose someone opens Powershell and types some command. Knowing only that powershell.exe started doesn't tell you everything typed inside that existing Powershell session.

Level 2: Process Command Line

Windows can enrich 4688 with the command line used to launch a new process. Conceptually: powershell.exe, versus: powershell.exe -Command "something"
However, this is still process creation telemetry.

Level 3: PowerShell Script Content

Powershell itself can provide additional telemetry about what it is processing. Today we introduce: 4104, PowerShell Script Block Logging

Why might a defender want both 4688 and Powershell-specific logging rather than choosing only one?

They give different layers of evidence. 4688 gives all sorts of process creation but command-line auditing can give more context into the usage.

Challenge 1: Improve CLIENT01's Visibility

Objective:

Improve CLIENT01 so that you can investigate:

1. The command line associated with new processes;
2. PowerShell script-block activity

Need to discover two policy settings: one conceptually means "include the command line when Windows records process creation." The other means: "Turn on PowerShell Script Block Logging."

Predictions:

GPO I should use: GPO-Workstation-Security-Auditing

Machine that receives: CLIENT01

Does 4688 already exist on CLIENT01: yes, we set this up already in a previous lab.

What additional information am I trying to add?

Path to find: Instead of advanced audit policy config, go to administrative templates and then system and then audit process creation.

For Powershell script block activity: also administrative templates, but then in Windows Components.

Self note: Policies would refresh automatically so often, so I wouldn't be forced with the command. In production, it is automated.

Challenge 2: Generate Benign Powershell Telemetry

Create several harmless PowerShell actions.

A: Simple command that retrieves harmless system information.

B: Pipeline.

C: Variable or small expression

Test A:

Command: whoami

Approx Time: 8/10/2026 1:46:24PM

Test B:

Command: Get-Service | Select-Object -First 5

Approx Time: 8/10/2026 1:47:20 PM

Test C:

Command: \$service = Get-Service

\$service.Count

Approx Time: 8/10/2026 1:48:32 PM